

CloudBees Agentic Software Engineer — Conversation Start / Take-Home Assessment

Timebox: ~2 hours of focused work. We'd rather see a small, thoughtful submission than a rushed ambitious one — if you hit the 2-hour mark, stop and write up what you have.

AI tools: You are encouraged to use a coding assistant (Claude, Copilot, Cursor, ChatGPT, etc.). The reflection on your experience using one is a graded part of this assessment, not an afterthought. In the follow-up interview we'll dig into what worked, what didn't, and where it helped or hindered.

API access: Use whatever model provider you already have access to (Anthropic, OpenAI, Google, a local model via Ollama, etc.).

Why we're asking you to do this

CloudBees is remote and async, so being able to demonstrate your thinking through code *and* writing matters. This exercise is a conversation driver for the next interview — we'll talk through your choices together. We're looking at:

- How you prototype agentic workflows with modern frameworks
- How you translate a fuzzy product concept into a working demo
- How you reason about **autonomy, reliability, and safety** when an LLM is making decisions
- How you collaborate with coding assistants
- How you address **observability and tracing gaps** within agentic flows
- How you reason about **evaluation methodology** and success metrics

We are explicitly **not** looking for production-grade systems, exhaustive test coverage, or polished UIs.

The task

1. Pick a framework

Use any agentic framework you like. Some options:

- [LangChain](#) / LangGraph
- [CrewAI](#)
- [Agn0](#)
- The Anthropic or OpenAI SDK directly (no framework is a valid choice)
- An MCP-enabled host (Claude Desktop, Goose, etc.)

If you pick something off this list, briefly justify it in your README. "I already know it" is a fine reason.

2. Build a small agentic flow

Build a demo where an agent has to **make a real decision** — not just call one tool and summarize the result. The agent should have to choose between options, recover from something going wrong, or branch based on what it observes.

Suggested scenarios (pick one or define your own):

- An agent that, given a GitHub repo URL, decides whether to investigate via the issues API, the commits API, or the README, depending on the question being asked — and explains its choice.
- An agent that reads a config file or piece of documentation, identifies an inconsistency or outdated value, and proposes a specific fix (with the agent deciding what counts as "outdated").
- An agent that has access to two or three tools where one is sometimes broken or returns bad data, and the agent has to detect this and fall back.
- A small "team" of agents *only if you can articulate why a single agent wouldn't work* — multi-agent for its own sake is an anti-pattern and we'd rather see you push back on the suggestion than build something elaborate.

What we want to see is the agent doing something an **if/else** couldn't easily do. If your demo would work the same with a hardcoded script, the scenario is too narrow.

3. Demonstrate one failure mode

In your code or README, show us **one thing that goes wrong** with your agent and how you handle (or would handle) it. Examples: the model hallucinates a tool argument, a tool times out, the agent loops, the output is unsafe to act on. A paragraph and a code snippet is enough — we mostly want evidence you thought about it.

Deliverables

Submit a link to a public repo (or a .tgz / .zip file) containing:

Code — your implementation. A single script, notebook, or small project is fine. Include a `requirements.txt` or equivalent so we can run it.

README.md — short and direct:

- What you built and why you picked this scenario (1–2 paragraphs)
- How to run it, including any env vars or API keys needed
- Assumptions, shortcuts, and known limitations
- The failure mode you identified and how you handled it
- Since we may not have access to your specific local model or API provider, please include a `sample_run.txt` or markdown file showing a complete, successful execution of your agent from start to finish

REFLECTION.md (or a section in the README) — 200–400 words on your experience using a coding assistant for this task:

- Did it meaningfully help with the agent's design, or mostly with boilerplate?
- Did it generate anything incorrect, unsafe, or misleading? What did you catch, and how?
- Where was it most useful? Where did you turn it off or ignore it?
- If you used it to write the agent itself (e.g., agent prompts), how did that feel — recursive, useful, weird?
- Outlining your strategy for maintaining traceability and system observability
- Evaluation Plan: Provide a 2–3 sentence overview of your systematic approach for assessing the agent's reliability and performance.

This section is graded equally with the code. Be specific; "it was helpful" is not an answer.

How we'll evaluate

Roughly weighted:

- **Reasoning** (40%) — does your scenario actually require agentic decision-making? Does your README explain *why* you made the choices you made?
- **Reflection** (25%) — depth and specificity about your AI-assisted workflow.
- **Reliability/safety thinking** (20%) — did you engage with what could go wrong?
- **Code quality** (15%) — does it run? Is it readable?

We are **not** weighting: framework choice, lines of code, number of tools, UI polish, or test coverage.

Submission

Send a link to your repo (or a branch) to your recruiter at least 24 hours before your next interview so we have time to read it.

 **Tip:** Pick the smallest scenario that lets you demonstrate real agentic reasoning. A 100-line demo with a thoughtful README beats a 500-line project we can't follow.